CS6008 Network Security — Assignment 2: DNS Security

Report

Team: Shiv Narang & Sabil

October 2026

Overview

A self-contained three-tier DNS hierarchy is built in Docker across three isolated subnets joined only by two no-NAT routers. We deploy plain DNS, attack it from an on-path position, then add defences (DNSSEC, then an encrypted last mile) one layer at a time and observe which attack each one stops.

Topology (fixed addresses). client `172.16.1.10`, webserver `172.16.1.20`, rogue-web `172.16.1.30`; resolver `172.16.2.10`, attacker `172.16.2.100` (P2–5) / `172.16.1.100` (P6–7); root-ns `172.16.3.10`, tld-ns `172.16.3.20`, domain-ns `172.16.3.30`; edge-router `172.16.1.2`/`172.16.2.2`, core-router `172.16.2.3`/`172.16.3.3`.

Shared foundations (`common/`). `base/` (the Phase 1 hierarchy and the all-purpose BIND 9.20 image), `signing/` (DNSSEC signing + trust anchor), and `interceptor/` (the C/libpcap attack engine). Each `phaseN/` directory is self-contained and reuses these.

Reproduced on: Colima (Docker 29, Ubuntu 24.04 VM, Apple Silicon). Every result below was produced by the `scripts/verify.sh` / `scripts/attack.sh` in the matching phase directory.

Phase 1 — Building the DNS Hierarchy

Design choices

- **Isolation by topology, not firewalling.** Each subnet is a Docker bridge with `internal: true` (no NAT, no Internet). A packet crosses subnets only through a router that is attached to *both* bridges; forwarding happens inside the router container's own network namespace, so Docker's host-level inter-bridge isolation never has to be fought. No other node forwards.
- **Static routes per node**, installed from `STATIC_ROUTES` by a tiny entrypoint (needs only `NET_ADMIN`): client → resolver-net via edge-router; resolver → auth-net via core-router and → client-net via edge-router; each nameserver → resolver-net via core-router. The routers need no extra routes (both their subnets are directly connected).
- **Real addresses end to end** — root hints and glue name the nameservers at their `172.16.3.x` addresses. Resolver is recursive with **DNSSEC validation off**. Names: root `a.root-servers.`, TLD `a.gtld-servers.`, domain `ns.example.com.`

1.3 Report answers

Resolved address, fetched page, isolation (commands + output).

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client dig +short www.example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i welcome
```

```
  <h1>Welcome to example.com</h1>
$ docker exec client dig +time=2 +tries=1 @172.16.3.10 example.com
;; no servers could be reached          # auth subnet not directly reachable
```

Root → TLD → Domain referral (from the core-router capture). A single `example.com/A` resolution crosses core-router as these exchanges (captured with `tcpdump -i any` on core-router, counted by destination):

```
16 queries -> 172.16.3.10 (root-ns)   # NS ? . / referral to com + glue resolution
 2 queries -> 172.16.3.20 (tld-ns)    # referral to example.com + glue ns.example.com
 2 queries -> 172.16.3.30 (domain-ns) # final A answer 172.16.1.20
```

On the wire, **everything is visible in cleartext**: the QNAME being asked, the 16-bit query ID and UDP source port that bind a reply to its query, the RR type, and the answer/authority/additional records (the referral NS names and their glue A records, then the final `example.com A 172.16.1.20`). (The root is contacted repeatedly only because our nameserver hostnames live in their own pseudo-TLDs; resolution still completes from the glue.)

How the resolver's query reaches root-ns (route + router).

```
$ docker exec resolver ip route get 172.16.3.10
172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10
```

The resolver has no interface on the authoritative subnet, so it uses the static route `172.16.3.0/24 via 172.16.2.3` and hands the packet to **core-router** (`172.16.2.3`), which forwards it out its `172.16.3.3` interface to root-ns.

What a node on the resolver's path could do. Because the whole exchange is unauthenticated cleartext, any node that sees it (e.g. an on-path box on the resolver subnet) can **read every query and forge a reply** — copy the QNAME, query ID and ports from the genuine response and substitute its own answer. We do exactly this in Phase 2.

Phase 2 — On-Path Response Forgery

Design choices

- **Diversion is a route on the router only.** `core-router` gets `ip route replace 172.16.2.10/32 via 172.16.2.100`, so the resolver's inbound responses are sent to the attacker first. The resolver and nameservers keep their Phase 1 configuration untouched.
- **The attacker does not forward.** It holds `NET_RAW` (+`NET_ADMIN`) and runs a C/libpcap program (`common/interceptor/dns_intercept.c`). The kernel would drop the diverted frames (their dst IP isn't ours and `ip_forward=0`), so we `pcap_open_live` to capture and `pcap_inject` to relay — the capture+inject path the brief requires.
- **Selective forge.** A BPF filter `ether dst <us> and udp and src port 53 and dst host 172.16.2.10` selects only diverted DNS responses (and never our own injected frames, whose dst MAC is the resolver's — so there is no loop). We rewrite the A rdata only when the question is the *exact* name `example.com` with `QTYPE=A`; IP + UDP checksums are recomputed and the packet length is unchanged.

2.2 / 2.3 Evidence

```
# relay mode (forging disabled) - transparent:
example.com     -> 172.16.1.20   ("Welcome to example.com")

# forge mode:
example.com     -> 172.16.1.30   (rogue "HACKED" page)
www.example.com -> 172.16.1.20   (untouched)
```

The attacker's own counters confirm it is in the path, relaying every packet and forging only the target answer.

2.4 Report answers

Why it works / the on-path advantage. Plain DNS has **no origin authentication**: a resolver accepts any reply that matches the outstanding query's QNAME, type, 16-bit ID, and UDP port tuple. An *off-path* spoofer must guess those (~ the ID × source-port space) and win a race against the real server. Our *on-path* attacker simply **reads** them from the genuine packet in transit and rewrites it, so there is nothing to guess and no race — success is deterministic.

How non-target names stay untouched. The interceptor parses the DNS question and only rewrites answers when the QNAME equals `example.com` exactly and the type is A. `www.example.com` is a different QNAME, so its packets fall through the `relay` path and are injected byte-for-byte — which is why it still returns `172.16.1.20`.

Phase 3 — DNSSEC Chain of Trust

Design choices

- **Algorithm ECDSAP256SHA256 (algo 13).** Small keys/signatures and fast validation; the modern default. Trade-off: shorter track record than RSA, and slightly less universal than it is more modern (vs Ed25519/algo 15); but it is the best-supported modern choice and keeps the signed zones compact.
- **Chain built child-first** (`common/signing/sign-all.sh`): sign `example.com`, put its KSK's DS in `com`, sign `com`, put its DS in the root, sign the root. The resolver anchors on the **root KSK only** (`trust-anchor.conf`, a `static-key`) and `dnssec-validation yes;`. Root hints unchanged from Phase 1.
- **The attacker and its Phase 2 binary are unchanged.** Only the zones and the resolver change.
- **No private key is in the repository** — keys and signed zones are build artifacts regenerated by the signer (git-ignored).

3.2 Verify (healthy signed hierarchy)

```
$ dig +dnssec example.com    ->  status: NOERROR; flags: qr rd ra ad ; + RRSIG
$ delv -a trust-anchor.conf @172.16.2.10 example.com
; fully validated
example.com.  3582  IN  A  172.16.1.20
```

3.3 Attack (re-run of Phase 2, unchanged)

```
$ docker exec client dig example.com
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL        # forged answer rejected
$ docker exec client dig +short www.example.com
172.16.1.20                                           # untouched name still validates
# resolver log:
validating example.com/A: no valid signature found
RRSIG failed to verify resolving 'example.com/A/IN': 172.16.3.30#53
# client fetch: (no page - resolution failed, attack defeated)
```

3.4 Report answers

What the resolver does / how it differs from Phase 2. The attacker injects the same forged A, but it cannot produce a matching **RRSIG** (it has no zone private key). The validating resolver follows the chain root KSK → DS(com) → com KSK → DS(example.com) → example.com ZSK and checks the signature over the A RRset; it finds *no valid signature*, cannot treat the name as secure, and returns **SERVFAIL** instead of the poison. In Phase 2 the same packet was accepted because nothing was checked.

What the on-path attacker can still read; what DNSSEC does / doesn't protect. On the wire the exchange is still cleartext — the attacker reads the QNAME, IDs and the answers exactly as in Phase 1 (now with RRSIG/DNSKEY/DS records visible too). DNSSEC provides **origin authentication and integrity** of

the DNS *data*; it does **not** provide confidentiality (queries/answers are still in the clear) and it does not protect the last hop to a non-validating stub (Phase 6).

Private keys in the repo? No. Only the signing *scripts* and the public trust anchor are committed; KSK/ZSK private keys are regenerated on each run.

Phase 7 — Encrypting the Last Mile (DoT / DoH) *(deploy half)*

> Our split: this half is the deployment + encrypted-resolution + "no cleartext" verification. The three attack attempts (7.3) build on the Phase 6 last-mile attacker and are Sabil's paired half.

Design choices

- **Private CA** (`pki/gen-ca.sh`): an RSA-2048 root CA issues the resolver a certificate whose CN/SAN is `resolver.cs6008.local` — **DNS SAN only, no IP SAN**. The CA is installed into the client's system trust store (`update-ca-certificates`); there is **no `-k`/insecure bypass**.
- **Reach by name, never by IP.** The client has `extra_hosts: resolver.cs6008.local -> 172.16.2.10` and connects to that name, so TLS is validated against the hostname.
- **Transport:** BIND `tls local-tls { cert-file; key-file; }` with `listen-on port 853 tls` (DoT) and `listen-on port 443 tls ... http` (DoH). Plain `:53` is left up so the (Sabil-side) downgrade attempt has a fallback to target. DNSSEC validation from Phase 3 still runs upstream.

7.2 Verify

```
# DoT:
;; TLS session (TLS1.3)-(ECDHE-X25519)-(RSA-PSS-RSAE-SHA256)-(AES-256-GCM)
;; status: NOERROR; Flags: qr rd ra ad ;  example.com. IN A 172.16.1.20
;; From 172.16.2.10@853(TLS)
# DoH:
;; HTTP session (HTTP/2-POST)-(resolver.cs6008.local/dns-query)-(status: 200)
;; From 172.16.2.10@443(HTTPS)
# validation really enforced (no bypass):
+tls-hostname=wrong.name.local  -> WARNING: TLS, handshake failed (Error in the certificate.)
missing CA file                 -> ERROR: failed to initialize crypto context
# no cleartext on the last mile (edge-router capture during a DoT lookup):
cleartext :53 packets = 0        encrypted :853 packets = 31
```

7.4 Report answers (our half)

DoT or DoH, and the trade-off. We deployed **both** and lead with DoT. DoT on its dedicated port 853 is simple and easy to audit — but equally easy to spot and block. DoH on 443 blends in with normal HTTPS and is far harder to block, at the cost of being harder to audit and heavier (HTTP/2 framing). Visibility vs. blend-in.

What an encrypted, authenticated last mile still does *not* protect. It secures only **client <-> resolver**. It does nothing for the resolver <-> authoritative path (still plain 53 upstream — DNSSEC covers integrity there, not confidentiality), it does not hide *which* resolver you use, and it does not stop a *malicious or mis-configured resolver* from lying to you — only client-side DNSSEC validation (Phase 6, Client B) catches that.

Phases 4, 5, 6 and the Phase 7 attack half *(Sabil)*

- **Phase 4 — Signature stripping & downgrade.** Extend `forge_or_relay()` in `common/interceptor/dns_intercept.` (see the `PHASE4` hook) with a strip mode.

- **Phase 5 — Zone walking.** NSEC walk on the Phase 3 signed zone; fix by re-signing with `NSEC3=1 sh run.sh` in `common/signing`.
- **Phase 6 — Last-mile forgery.** Same interceptor, diversion on the edge-router, victim = client (Client A stub vs Client B validating).
- **Phase 7 (attack half)** — the three attempts × two clients.

(Sabil to complete and write up.)

Appendix — Generative AI usage

This assignment was built with AI assistance (Claude Code). How it was used and verified:

- **What AI did:** scaffolded the repo; wrote the Docker Compose topologies, BIND zone/daemon configs and routing/entrypoint scripts; wrote the DNSSEC signing scripts; wrote the C/libpcap interceptor (`dns_intercept.c`); wrote the Phase 7 PKI and DoT/DoH configs; and drafted this report from captured output.
- **Representative prompts:** "divide this 7-phase DNS-security assignment into an equal split"; "build the Phase 1 base — compose, zones, routing"; "write the DNSSEC signing scripts and a validating resolver"; "write the C/libpcap on-path interceptor with a selective A-record forge"; "stand up the Phase 7 private CA + DoT/DoH and verify no cleartext".
- **How every output was verified:** nothing was taken on trust — each phase was deployed on Docker (Colima) and checked against the brief's own success criteria. Phase 1: resolution, web fetch, isolation ("no servers could be reached"), and a core-router referral capture. Phase 2: relay transparency, then forge → rogue with `www` untouched. Phase 3: `ad` flag + `delv` "fully validated", then the re-run attack → **SERVFAIL** with the matching resolver log line. Phase 7: DoT/DoH resolution, deliberate **negative** cert tests (wrong hostname / missing CA both fail), and a `tcpdump` on edge-router showing zero cleartext `:53`. All commands are in each phase's `scripts/`.